Architetture e sistemi operativi

Contents

1 Introduzione **4**

2 Processore, memoria e istruzioni **5**
2.1 Processore . . . 5
2.2 Memoria . . . 5
2.3 Tipi di istruzioni . . . 5

3 Livelli di astrazione dei calcolatori **7**
3.1 Principi importanti . . . 7

4 Circuiti Digitali **9**
4.1 Sistema binario . . . 9
4.2 Numeri negativi . . . 9
4.3 Numeri con la virgola (Floating Point) . . . 10
4.4 Caratteri (ASCII) . . . 10
4.5 Notazione per grandezza input/output . . . 11
4.6 Espressioni dei circuiti . . . 11
4.7 Tempi per calcolare risultati . . . 11
4.8 Esadecimale . . . 12
4.9 Processo di progettazione dei circuiti digitali . . . 13
4.10 Mappe di Karnaugh . . . 14
4.11 Disturbi aleatori (ALEA) . . . 15

5 Verilog **16**
5.1 Dichiarazione moduli . . . 16
5.2 Modulo tester . . . 17
5.3 Monitor . . . 17
5.4 Full Adder . . . 17
5.5 Tools . . . 17

6 Moduli **18**
6.1 Sommatore . . . 18
6.2 Comparatore . . . 18
6.3 Codificatore . . . 18
6.4 Moduli per algebra booleana . . . 19
6.5 Full Adder . . . 20
6.5.1 Full Adder in complemento a 2 . . . 21
6.6 Multiplexer . . . 21
6.7 Demultiplexer . . . 22

7 Memoria **24**
7.1 Registri . . . 24
7.2 SR Latch . . . 24
7.3 D Latch . . . 25
7.4 Flip Flop D . . . 26
7.5 Segnale di reset . . . 27

7.6 Reti sequenziali 27

8 Esercizi **29**
8.1 Cosa calcola una rete 29
8.1.1 Descrizione 29
8.1.2 Soluzione 29
8.2 Circuito di una sveglia 30
8.2.1 Descrizione 30
8.2.2 Soluzione 30

1 Introduzione

In questo corso andremo a:

- Capire come funziona un elaboratore.
- Capire quali tipi di istruzioni è in grado di eseguire un elaboratore nativamente.
- Come lo si può dotare di un sistema operativo.

Il corso utilizza Windows e Linux come esempi di sistemi operativi.

Codice Teams: 6uk5ntg

Email Docente: marco.delutto@unipi.

2 Processore, memoria e istruzioni

2.1 Processore

Il **processore** è in grado di fare operazioni piccole.
Il processore contiene i **registri**, memorie piccolissime e velocissime utilizzate per memorizzare dati temporanei legati alle istruzioni da eseguire. Ecco alcuni registri speciali:

- **Program Counter (PC)**: un registro speciale che conserva lo stato di esecuzione memorizzando l'indirizzo della prossima istruzione da eseguire.
- **Instruction Register (IR)**: contiene l'istruzione prelevata dalla memoria che deve essere eseguita.
- **Stack Pointer (SP)**: contiene la posizione attuale dello stack.

2.2 Memoria

Il processore è unito ad una **memoria**, costituita da **celle**, è vista come un vettore:

$$M[i] = k$$

Ogni cella ha un valore e la sua capacità è 1 parola (32 o 64 bit).
Una volta che il processore viene acceso, esso esegue:

```
while(true):
    instr = getInstructionFrom(M)
    decoded = decode(instr)
    execute(decoded)
    determineResults()
    ...
```

Il processore quindi:

- Estrae istruzioni dalla memoria
- Le decodifica
- Le esegue
- Calcola i risultati

2.3 Tipi di istruzioni

Le istruzioni possono essere:

- Operative
  - Aritmetiche
    - ADD
    - SUB
    - MUL

        * DIV
    - Logiche
        * AND
        * OR
        * NOT
    - Shift / rotazione
        * SHIFT
        * ROT
    - ...
- Di Memoria
    - LOAD
        * Restituisce un valore da un indirizzo:

$$\text{LOAD val, addr}$$

    - STORE
        * Salva un valore in un indirizzo:

$$\text{STORE val, addr}$$

- di Salto
    - "Saltano" pezzi di codice, volendo anche secondo istruzioni, aggiornando il PC alla corrispondente istruzione.

```
inizio:
\dots
jmp fine:
-skipped code-
fine:
```

3 Livelli di astrazione dei calcolatori

Per capire come opera un computer osserviamo dei livelli di astrazione, sviluppati a "torre" nel seguente modo:

applicazioni
sistema operativo
architettura (set istruzioni)
μ - architettura (processore)
logica (componenti)
circuiti digitali (0/1, 5V/0V)
circuiti analogici
dispositivi
fisica

Non andremo a vedere niente delle caratteristiche hardware degli ultimi 3 livelli, non ci interessa.
Per capire:

- Noi partiamo col realizzare dei circuiti digitali che servono per creare dei componenti
- Tali componenti ci servono per realizzare un processore
- Al processore dovremo dare in pasto delle istruzioni
- Le istruzioni servono al sistema operativo e alle applicazioni

Tra i livelli di "architettura" e "sistema operativo" si trova un **interfaccia**, genericamente attribuita al livello **i**. Questo significa che:

- Il livello del sistema operativo corrisponde al livello **i** (si trova al livello dell'interfaccia)
- Il livello dell'architettura corrisponde al livello **i - 1**

Al livello **i + 1**, tra sistema operativo e applicazioni, si trovano le **chiamate al sistema operativo**, chiamate **API**.

Ogni livello conosce solo il **livello inferiore** a se, questo è per mantenere i livelli superiori attraverso diversi livelli inferiori; per esempio un telefono Samsung e un Motorola sono realizzati in maniera diversa, ma entrambi utilizzano Android, questo è possibile proprio grazie al fatto che i livelli non si concentrano sui livelli soprastanti.
Un altro esempio: non si possono portare app iOS su Android.
Spesso esistono dei traduttori che permettono di convertire istruzioni (x64,x86) per permettere compatibilità con sistemi operativi e processori differenti.

3.1 Principi importanti

I componenti che svolgono le istruzioni seguono questi concetti:

- **Gerarchia**: il principio dei livelli, ogni livello utilizza i componenti dei livelli sottostanti senza occuparsi di cosa fanno.

- **Modularità**: presenza di moduli distinti per la specializzazione e composizione del lavoro.
- **Regolarità**: si realizzano strutture simili e ripetibili invece di fare sempre le cose in modo diverso.

Si usano moduli più piccoli per svolgere azioni più grandi quando combinati.

4 Circuiti Digitali

4.1 Sistema binario

Noi utilizziamo il sistema decimale, ovvero un sistema **posizionale**, dove ogni cifra possiede un valore diverso a seconda della sua posizione.
Anche il **sistema binario** è un sistema posizionale, ma le cifre sono solo 2 (0,1).

- **Esempio decimale:**
$$13_{10} = (1 \cdot 10^1) + (3 \cdot 10^0)$$
- **Esempio binario:**
$$10_2 = (1 \cdot 2^1) + (0 \cdot 2^0) = 2_{10}$$

Nei computer si utilizza il sistema binario per via della sua semplicità e della sua compatibilità con i moduli logici, per esempio una moltiplicazione è più facile da svolgere in binario tramite un modulo **AND**, invece che in decimale.

4.2 Numeri negativi

Per rappresentare i numeri negativi in binario ci sono due modi:

- **Modulo e segno:** si riserva il **MSB** (Most Significant Bit, più a sinistra) per il segno (1 se negativo, 0 se positivo).
- **Complemento a 2:** nato perche complicate le operazioni con modulo e segno (if else); si ha un algoritmo per ottenere il binario di un numero negativo:
  1. Si trasforma il numero in binario.
  2. Si fa il NOT di ogni bit
  3. si aggiunge "+1"

Su un byte di solito si possono realizzare **256** possibili combinazioni positive, ma col complemento a 2 l'intervallo si spezza:

- 128 valori positivi (0 - 127)
- 128 valori negativi (-128 - -1)

Nel modulo e segno addirittura ci sono 2 rappresentazioni diverse per lo 0, altro motivo per cui è stato introdotto il complemento a 2.
Ecco come funziona veramente il complemento a 2:

- Il MSB vale un valore negativo (-2^7)
- Tutti gli altri bit valgono valori positivi
- Se si ricalcola la somma di un complemento a 2 per trovare il suo valore decimale, si ottiene il numero negativo.

Per esempio, calcoliamo -10:

$$10_{10} = 00001010_2$$

$$!(00001010) = 11110101$$

$$11110101 + 1 = 11110110$$

$$11110110_2 = (-2^7 + 2^6 + 2^5 + 2^4 + 2^2 + 2)_{10} = (-128 + 64 + 32 + 16 + 4 + 2) = -10_{10}$$

ATTENZIONE: se si effettua una somma tra numeri in complemento a 2 che hanno segni concordi e si ottiene un risultato con segno diverso allora significa che c'è stato un **overflow**.

4.3 Numeri con la virgola (Floating Point)

Il **floating point (FP)** permette di rappresentare i numeri con cifre decimali.
Ecco la sua struttura:

[segno][esponente][mantissa]

Questa struttura può essere lunga 32 o 64 bit.

- **Segno**: 1 bit, segno (+/-) del numero.
- **Esponente**: 8 o 11 bit, esponente per cui "10" è moltiplicato il numero decimale, per esempio:
$$10,5 = 105 \cdot 10^(- 1)$$
- **Mantissa**: 23 o 52 bit, contiene il numero intero da moltiplicare al "10" ().

4.4 Caratteri (ASCII)

Per rappresentare i caratteri in binario si utilizza la **codifica ASCII**, che utilizza **ottetti** (7/8 bit) per rappresentare ogni carattere.
Ogni carattere è identificato da un **ID**. Offre:

- 26 lettere maiuscole
- 26 lettere minuscole
- 10 cifre
- Punteggiatura
- Caratteri speciali (invio, tab, DEL)

Vediamo i range di ID che ci interessano:

- **Non stampabili:** $0 - 31, 127$
- **Stampabili:** $32 - 126$
- **Cifre:** $48 - 57$
- **Lettere maiuscole:** $65 - 90$

- **Lettere minuscole:** $97 - 122$
- **Spazio:** 32

Come si può notare, ogni lettera maiuscola è sempre alla stessa distanza dalla sua corrispondente minuscola: per questo basta aggiungere "**32**" ad una lettera maiuscola (o sottrarlo alla minuscola) per switchare tra lowercase e uppercase.

4.5 Notazione per grandezza input/output

Per indicare quanti bit è grande un input o output, si usa questa notazione:

$$n$$

$$\nrightarrow$$

4.6 Espressioni dei circuiti

Per rappresentare i circuiti si usano delle **espressioni**, vediamo delle notazioni di algebra booleana in tale progettazione:

- Quando un parametro ha un "overline" come $\overline{x}$ allora significa che passa da un NOT.
- La moltiplicazione tra variabili corrisponde all'AND, dato che se facciamo la moltiplicazione tra più valori booleani il risultato sarà diverso da 0 solo se tutti i membri sono settati a "1".
- La somma tra variabili corrisponde all'OR, dato che il risultato è non nullo solo se almeno uno dei membri è settato a "1".

Nota bene: un qualsiasi valore positivo, anche se diverso da "1", viene contato come **true**. Vediamo un esempio:

$$\overline{x}yz + x\overline{y}z + xy\overline{z} + xyz$$

Questo è un circuito con 4 moduli AND, 3 moduli OR e 3 moduli NOT realizzato in questo modo:

4.7 Tempi per calcolare risultati

Quando realizziamo circuiti digitali bisogna tenere di conto quanto ci mettono le porte a inizializzarsi e quanto ci mette il risultato a essere emesso.
Indichiamo il ritardo di una porta da massimo **8 ingressi** con:

$$\Delta t$$

Si considera il ritardo per le porte di AND e OR, il tempo della porta NOT è talmente insignificante che non viene considerato ($= 0$).
Nei nostri circuiti si usano sempre porte da 8.
Per realizzare porte da 9 o più è necessario collegare più porte insieme e farle congiungere ad una porta unica.
Quando bisogna realizzare circuiti con tanti ingressi si tende a realizzare una struttura ad albero.
Ecco come si trova il tempo di impiego di un modulo:

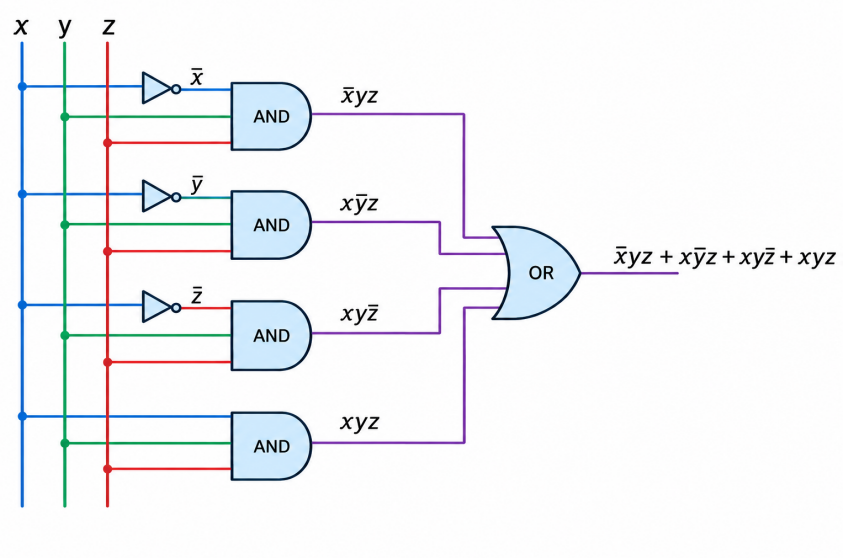


Figure 1: Circuito logico

1. Si crea la tabella di verità con a sinistra gli input e a destra gli output
2. Si scrivono tutte le righe possibili degli input (combinazioni, 2^n, n = numero di input diversi)
3. Si calcola il risultato a seconda del modulo e si scrive per ogni combinazione
4. Ripetere l'istruzione precedente per ogni output
5. Una volta trovata la tabella completa, si trova l'espressione per calcolare ogni output: si eguaglia un output alla somma dei prodotti dei vari input delle combinazioni che hanno risultato un output = "1" (espressione logica, i bit = 0 sono "negati", ovvero con la overline (NOT))
6. Si contano termini (OR) e variabili (AND), spesso ogni modulo li usa entrambi, arrivando al minimo a $2\Delta t$, però se uno tra i due supera il numero 8 allora si conta un tempo in più.

Spesso le formule si possono semplificare, ma ciò non influisce il tempo impiegato dai moduli, bensì ci può permettere di risparmiare materiale e spazio.

4.8 Esadecimale

Oltre al sistema binario, spesso viene utilizzato anche il sistema **esadecimale**, ovvero in base 16.
Questo sistema è comodo, basta pensare al fatto che 16 in binario necessita di esattamente 4 bit (un **nibble**), questo vuol dire che ogni 4 bit si ha una cifra in esadecimale e viceversa.
Ecco le cifre:

$$1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E, F$$

Con:

$$A = 10, F = 16$$

La base esadecimale viene introdotta da un "**0x**", così:

$$0x5AF$$

4.9 Processo di progettazione dei circuiti digitali

Ecco il procedimento:

1. Descrizione a parole della richiesta
2. Realizzazione pseudo codice
3. Approccio strutturale, composizione componenti
4. Tabella di verità
5. Somma di prodotti (espressione)
6. Rete di porte
7. Tempo di stabilizzazione

Esempio : dato x da n bit:

```
if(pari(x)) {
    x++;
} else {
    x--;
}
```

Per trovare i pari basta notare che tutti i numeri pari in binario hanno il LSB (Least Significant Bit) segnato a "0".

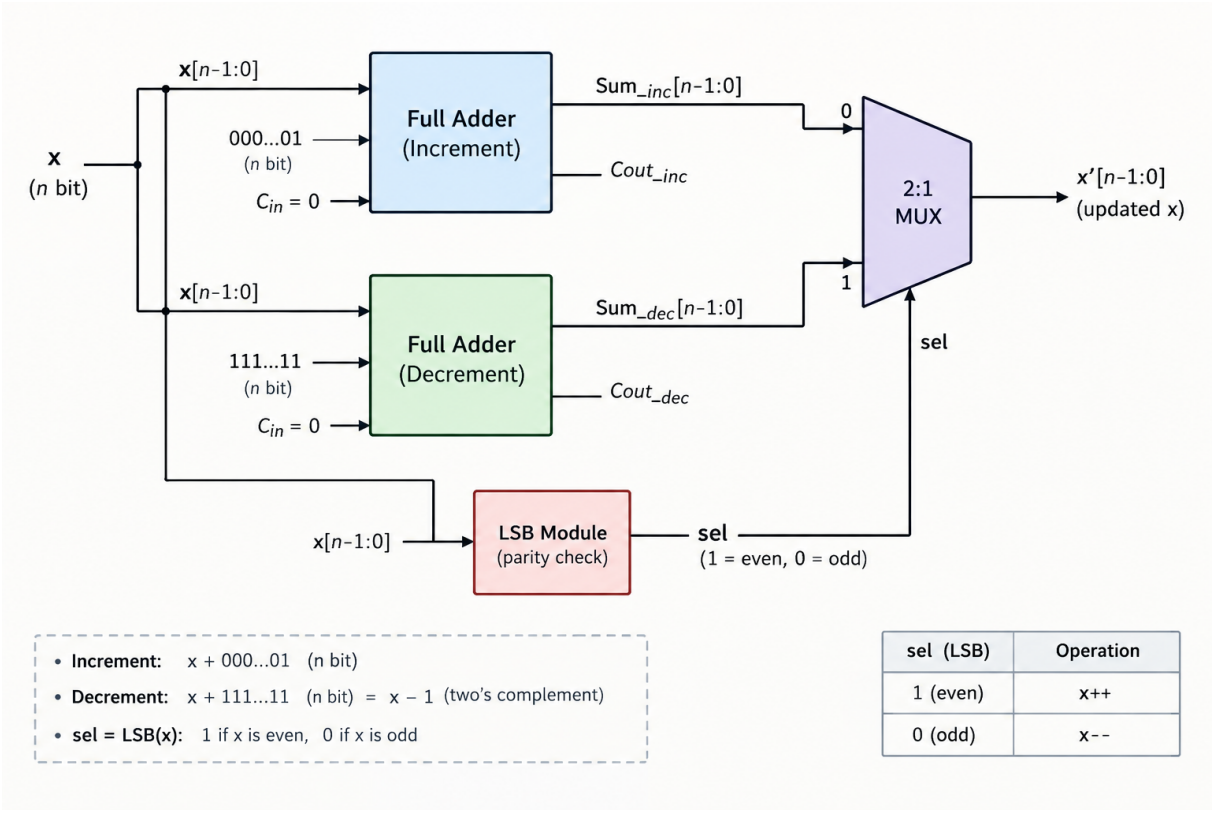


Figure 2: Circuito Pari-Dispari

4.10 Mappe di Karnaugh

Modo alternativo alle tabelle di verità, permette di semplificare notevolemnte le espressioni logiche, si rappresentano cosi:

	$x=0$	$x=1$
$y=0$	0	0
$y=1$	0	0

	$x_1x_0=00$	$x_1x_0=01$	$x_1x_0=11$	$x_1x_0=10$
$y_1y_0=00$	0	0	0	0
$y_1y_0=01$	0	0	0	0
$y_1y_0=11$	0	0	0	0
$y_1y_0=10$	0	0	0	0

Come si può vedere viene prima "11" che di "10" perche si usa il **codice Gray**, il quale indica che ad ogni passaggio (celle adiacenti) può cambiare un solo bit.

Una volta che abbiamo fatto la tabella, si **raggruppano gli 1** adiacenti che formano un gruppo di grandezza pari ad una potenza di 2, scegliendo quello più grande (aiuta a ridurre di più).
Per esempio:

	$x_1x_0=00$	$x_1x_0=01$	$x_1x_0=11$	$x_1x_0=10$
$y_1y_0=00$	1	1	0	0
$y_1y_0=01$	1	1	0	0
$y_1y_0=11$	0	0	0	0
$y_1y_0=10$	0	0	0	0

L'espressione in questo caso sarebbe:

$$F = \overline{x_1}\,\overline{x_0}\,\overline{y_1}\,\overline{y_0} + \overline{x_1}\,x_0\,\overline{y_1}\,\overline{y_0} + \overline{x_1}\,\overline{x_0}\,\overline{y_1}\,y_0 + \overline{x_1}\,x_0\,\overline{y_1}\,y_0$$

Le prime 4 celle possono essere raggruppate.

ATTENZIONE: anche le celle opposte sono considerate adiacenti, come se la tabella fosse arrotolata.
Una volta trovato il gruppo, costruiamo l'espressione semplificata seguendo queste regole:

- Se nell'analisi degli "1" una variabile **cambia**, allora viene **esclusa**.
- Se invece rimane costante, viene **tenuta**.

Nell'esempio precedente:

- x_1 non cambia, rimane.
- x_0 cambia, viene scartata.
- y_1 non cambia, rimane.
- y_0 cambia, viene scartata.

L'espressione adesso è quindi:

$$F = \overline{y_1 x_1}$$

4.11 Disturbi aleatori (ALEA)

Quando si effettuano le transizioni da un valore all'altro tramite delle porte logiche, le quali hanno dei ritardi, possono crearsi delle variazioni indesiderate temporanee, ovvero sbalzi di valore spontanei che possono provocare glitch.
Per esempio, se abbiamo:

$$F = AB + \overline{A}C$$

Se il valore di A cambia da 0 a 1, il ritardo delle porte potrebbe non propagare il cambiamento di valore al secondo modulo dell'espressione in tempo, generando un calcolo errato.
Per risolverli, si modifica la rete logica e si introducono termini ridondanti, i quali nella mappa di Kernaugh "allacciano" insiemi separati.
Più formalmente, ogni coppia di gruppi adiacenti nella tabella di Kernaugh devono essere collegati da un terzo gruppo, come un "ponte".

Applicare queste modifiche di rete sicuramente aumenta la complessità e il tempo impiegato, ma in questo modo ci assicuriamo che la rete funzioni senza errori.

5 Verilog

Verilog è un **HLD (Hardware Descritpion Language)**, ovvero un linguaggio di programmazione che descrive i circuiti logici.
Utilizzeremo:

- iverilog: compilatore Verilog.
- vvn: interprete (genera dumptest).
- gtkwave: mostra le onde grafiche (GUI).

Questi comandi vanno utilizzati in sequenza su un file Verilog per testarlo ed eseguirlo.

5.1 Dichiarazione moduli

Si ha in input uno schema di un circuito scritto con Verilog (.v), il quale viene testato da un altro file / funzione "test"(.v).
Viene generato un file di estensione ".vcd" che contiene i risultati.
Ogni pezzo di circuito deve essere rappresentato come **modulo**, vediamo come creare un MUX:

```
primitive mux(output z, input x, input y, input c);
table
    1 ? 0 : 1;
    ? 1 1 : 1;
    0 ? 0 : 0;
    ? 0 1 : 0;
endtable
endprimitive
```

Per convenzione, si mettono prima gli output e poi gli input.
Da standard, input e output specificati nel modo soprastante sono di **1 bit**.
Dopo è stata specificata una **tabella**, la quale corrisponde proprio alla tabella di verità (sintetizzata in questo caso). I "?" indicano un valore qualsiasi.
Giustamente, la tabella e il primitive vanno chiusi.
Primitive però non si usa mai, si usa **module**:

```
module mux (...)
```

Per calcolare l'**espressione logica** si dichiara un modulo con **module** e poi si fa cosi:

```
module mux(output z, input a, input b, input c);
assign
    z = b && c || a && !c
```

Si può utilizzare il bit di controllo per scegliere l'output:

```
module mux(output z, input a, input b, input c);
assign
    z = (c == 0 ? a : b)
```

5.2 Modulo tester

Ecco come si testa (esegue) il circuito progettato:

```
(circuito)
\dots
module test():

    reg ix, iy, ic;
    wire out;

    mux istanzaMux(out, ix, iy, ic);

    integer i;

    initial
        begin
            $dumpfile("test.vcd");
            $dumpvars
            for(i=0;i<8;i=i+1)
```

Si apre la funzione "test" che contiene le istruzioni del run-time.
Si istanziano le variabili necessarie e i moduli necessari.

5.3 Monitor

Questo comando restituisce i risultati specificati ogni volta che viene passato un input e stampa un messaggio custom:

```
(tester)
\dots
$monitor($time," in = %b %b controllo = %b ==> z = %b", ina, inb, inc,
...
```

5.4 Full Adder

```
module far(output c, output z, input x, input y, input r);
    far ris(z,x,y,r)
    far car(c,x,y,r)
```

5.5 Tools

Ecco alcuni strumenti importanti per lavorare con Verilog:

- **Quartus Prime Lite Edition:** rappresentazione grafica dei moduli utilizzati in un file Verilog.

6 Moduli

Questa sezione elenca vari esempi di moduli per le operazioni del calcolatore.

6.1 Sommatore

Semplice modulo che:

- Prende in input due numeri (1 bit) ed un riporto iniziale di "0" (non lo considera).
- Li somma e restituisce il risultato in output.
- Se c'è overflow, setta a "1" il secondo output che indica il riporto.

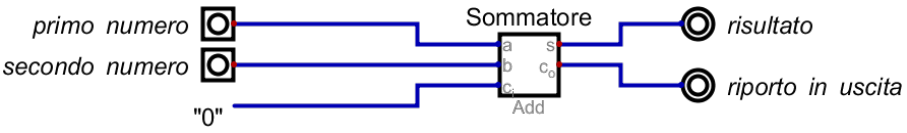


Figure 3: Sommatore

Un sommatore a più bit è costruito nel solito modo.

6.2 Comparatore

Modulo che prende due numeri in input (a, b) e restituise 3 output:

- $G = \begin{cases} 1, a\ b \\ 0, \text{ altrimenti} \end{cases}$
- $E = \begin{cases} 1, a == b \\ 0, \text{ altrimenti} \end{cases}$
- $L = \begin{cases} 1, a \leq b \\ 0, \text{ altrimenti} \end{cases}$

Tempo d'esecuzione: $2\Delta t \cdot N$ con N = numero di bit dei numeri, però non può scendere sotto $3\Delta t$.

6.3 Codificatore

Ha un input e un output, restituisce la posizione dell'1 nell'input. Denotato da una scatola con "COD" scritto dentro.

6.4 Moduli per algebra booleana

Per svolgere i controlli booleani si utilizza una combinazione (circuito) dei seguenti moduli:

- Modulo NOT



Figure 4: Modulo NOT

- Modulo AND

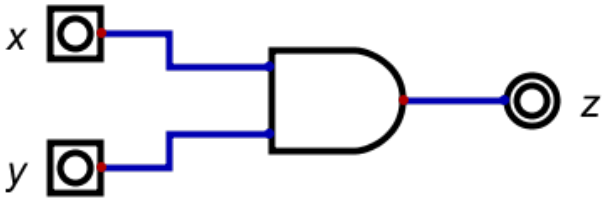


Figure 5: Modulo AND

- Modulo NAND

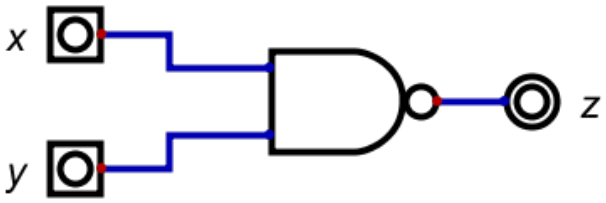


Figure 6: Modulo NAND

- Modulo OR

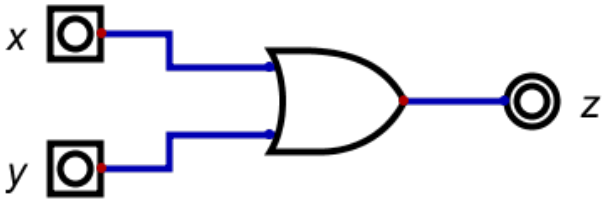


Figure 7: Modulo OR

- Modulo NOR

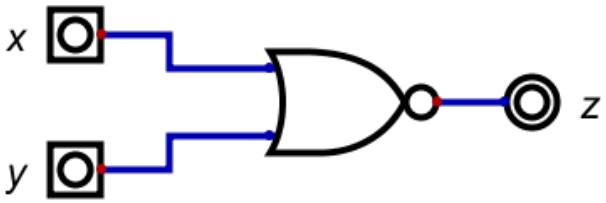


Figure 8: Modulo NOR

- Modulo XOR

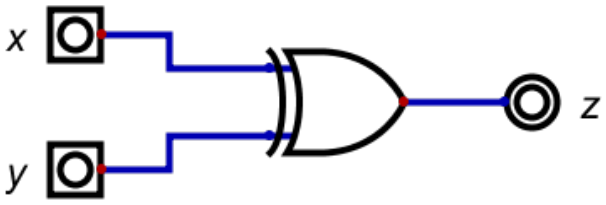


Figure 9: Modulo XOR

- Modulo XNOR

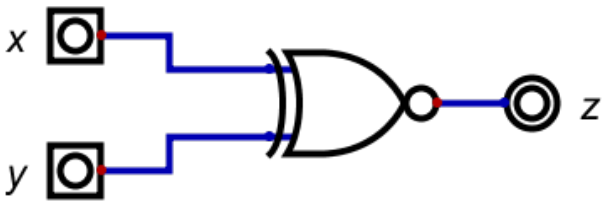


Figure 10: Modulo XNOR

Tutti prendono due o più input in entrata e restituiscono un output ad eccezione del NOT, che prende un solo input.

6.5 Full Adder

Sommatore speciale che:

- Ha 2 input
- Ha 1 output risultato
- Ha 1 output di riporto (che va al prossimo FA, messi in sequenza per sommare piu numeri)
- Ha 1 riporto iniziale (anche nullo)

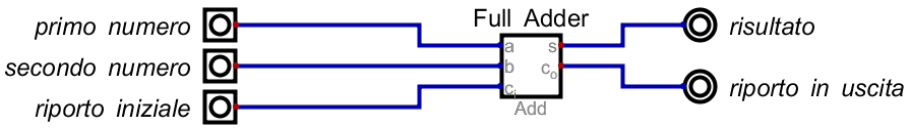


Figure 11: Full Adder

X	Y	C_{in}	Ris	Rip
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

Table 1: Tabella di verità del Full Adder

$$Ris = X \cdot \overline{Y} \cdot \overline{C_{in}} + \overline{X} \cdot Y \cdot \overline{C_{in}} + \overline{X} \cdot \overline{Y} \cdot C_{in} + X \cdot Y \cdot C_{in}$$

$$Rip = X \cdot Y + X \cdot C_{in} + Y \cdot C_{in}$$

Tempo d'esecuzione: $2\Delta t \cdot N$ con N = numero di bit dei numeri.

6.5.1 Full Adder in complemento a 2

Un Full Adder può sommare numeri in complemento a 2 (negativi) con queste particolari modifiche:

- L'input da portare in complemento a 2 viene negato
- Il riporto iniziale è necessariamente uguale a 1

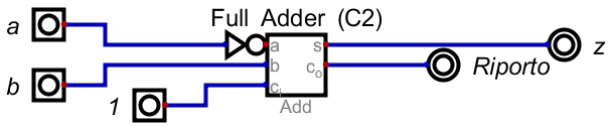


Figure 12: Full Adder (C2)

Tempo d'esecuzione: $2\Delta t \cdot N + \Delta t$ con N = numero di bit dei numeri.

6.6 Multiplexer

Il **multiplexer** è un modulo molto importante che sceglie uno tra due input in base ad un input di controllo che indica la porta dell'input da restituire.

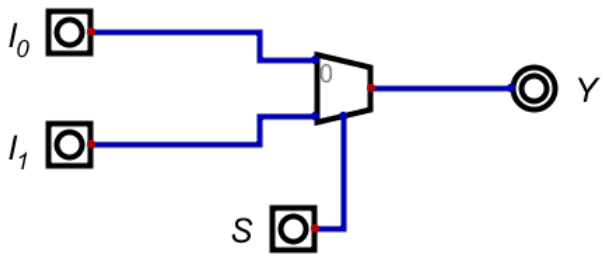


Figure 13: Multiplexer

I_0	I_1	S	Y
0	0	0	0
0	1	0	0
1	0	0	1
1	1	0	1
0	0	1	0
0	1	1	1
1	0	1	0
1	1	1	1

Table 2: Tabella di verità del Multiplexer 2:1

$$Y = \overline{S} \cdot I_0 + S \cdot I_1$$

In realtà il multiplexer può avere più input, ma questo prevede l'espansione della grandezza dell'input di controllo.

Tempo d'esecuzione: $2\Delta t$ se usiamo MUX 2:1 e li mettiamo paralleli, $2\Delta t \cdot \log_8 N$ se usiamo MUX con ingressi multipli.

6.7 Demultiplexer

Contrario del multiplexer, ha 1 input e 2 porte in uscita.
Trasmette l'input in una porta in uscita a seconda del **bit di controllo** in input e azzera la porta in uscita non utilizzata.

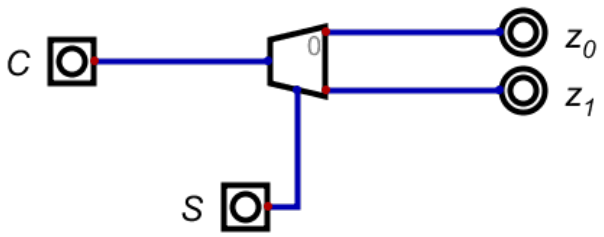


Figure 14: Demultiplexer

C	S	z_1	z_0
0	0	0	0
0	1	1	0
1	0	0	0
1	1	0	1

Table 3: Tabella di verità del Demultiplexer 1:2

$$z_1 = C \cdot \overline{S}$$

$$z_0 = C \cdot S$$

Impiega un tempo di Δt (fa solo AND).
Anche questo, come il multiplexer, può avere più porte in uscita, ma prevedono appunto l'espansione dell'input di controllo.

7 Memoria

7.1 Registri

Il registro funziona cosi:

- Input x di n bit
- Output y di n bit
- Clock in input
- Reset in input (Per quando bisogna fare il RESET) (non presente in figura)
- **Write Enable** in input (dice se scrivere o meno)

$$\begin{cases} clock == 1 \&\& we = 1 \implies \text{ricordati } x \\ \text{altrimenti ricordati il valore precedente} \end{cases}$$

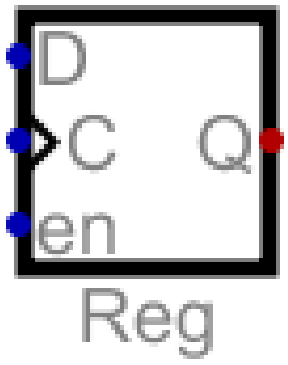


Figure 15: Registro

7.2 SR Latch

Abbiamo un modulo che ha:

- Input **SET**
- Input **RESET**
- Write Enable in input (Non presente in figura)
- Output su 1 porta tra 2 (Q)

$$\begin{cases} SET == 1 \implies \text{ricordati } 1 \\ RESET == 1 \implies \text{ricordati } 0 \end{cases}$$

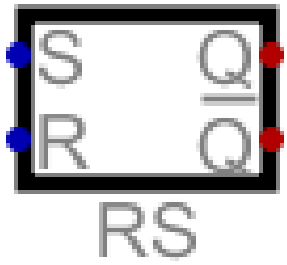


Figure 16: SR Latch

Per realizzare questa cosa bisogna:

- Collegare una porta OR Negata a ogni input (1 ciascuno)
- L'output di ognuno ritorna alla porta OR Negata come secondo input di tale porta dell'altro.

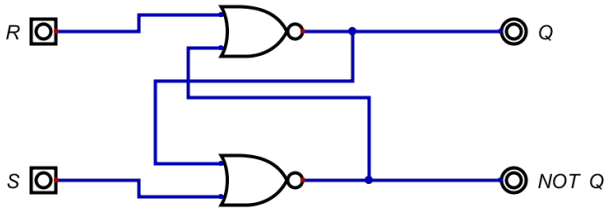


Figure 17: Circuito SR Latch

Ecco la tabella di verità:

S	R	Q	$\overline{Q}$	Stato
0	0	$Q(t)$	$\overline{Q}(t)$	Mantiene
0	1	0	1	Reset
1	0	1	0	Set
1	1	0	0	Vietato

Table 4: Tabella di verità del latch SR con porte NOR

Si crea un problema se entrambi SET e RESET sono settati a 1 perche si crea questo:

$$Q = 0 = \overline{Q}$$

7.3 D Latch

Per risolvere il problema di SR latch, si introducono due input:

- **EN**: dice quando scrivere (sensibile al livello), talvolta viene scritto **CLK**.
- **D**: dice cosa scrivere.

Ecco lo schema interno di tale modulo:

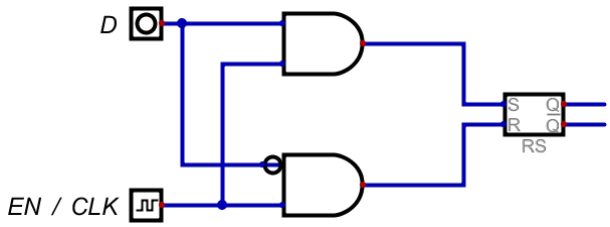


Figure 18: Circuito interno di D Latch

Di solito però viene rappresentato in questo modo:

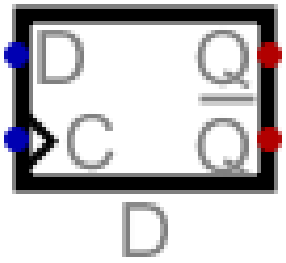


Figure 19: D Latch

Sembra simile all'SR Latch, ma utilizzando un paio di questi possiamo realizzare un **Flip Flop D**.

7.4 Flip Flop D

Un **Flip Flop D** può essere realizzato mettendo in sequenza due D Latch in questo modo:

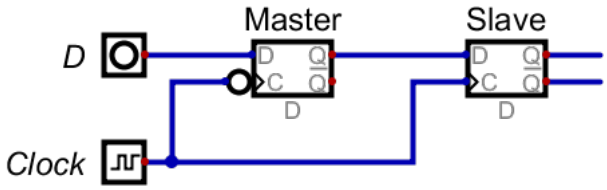


Figure 20: Flip Flop D: modello Master-Slave

Il primo D Latch si chiama **master** (prepara il dato) e il secondo **slave** (trasferisce all'uscita).
I due D Latch sono sensibili al livello, ad uno stato scrive il master, all'altro scrive lo slave.
Tutto il circuito intero, ovvero il **flip flop**, è invece sensibile al fronte del clock, ovvero quando si deve effettuare il cambio tra master e slave.
Se vogliamo, possiamo includere un Write Enable che gestisce i tempi di attivazione:

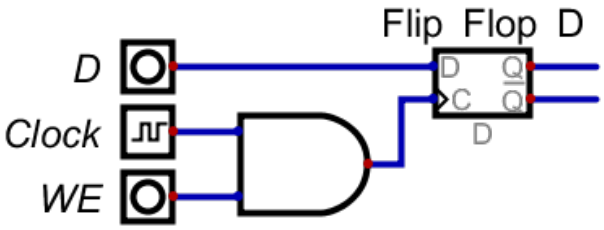


Figure 21: Flip Flop D: Write Enable integrato

7.5 Segnale di reset

Per riportare a "0" un valore di un registro si usa un segnale di **reset**.
Esso viene portato in input in un registro.
La sua versione sincrona è dipendente dal fronte del clock.

7.6 Reti sequenziali

Con i registri si possono sviluppare le **reti sequenziali** per rappresentare **automi**.
Supponiamo di avere una sequenza di bit qualsiasi, bisogna svolgere una sequenza di bit risultante che immette:

$$\begin{cases} 1 : \#1 = \text{pari} \\ 0 : \text{altrimenti} \end{cases}$$

Si realizza prima un automa, in questo caso di **Moore**, ovvero che l'output è associato allo stato:

- S0: "1" Dispari
- S1: "1" pari

$$\begin{cases} S0 \to 0 \implies S0 \\ S0 \to 1 \implies S1 \\ S1 \to 0 \implies S1 \\ S1 \to 1 \implies S0 \end{cases}$$

Ecco il diagramma dell'automa:

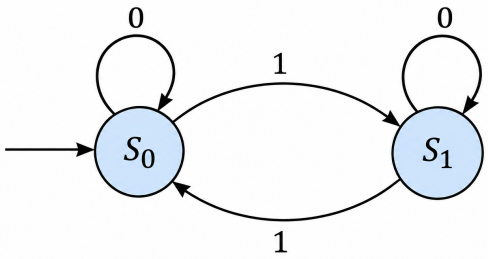


Figure 22: Automa

La tabella di verità è quindi:

Stato attuale (S)	Input(I)	Stato successivo (S^I)
0	0	0
0	1	1
1	0	1
1	1	0

Vediamo ora l'espressione logica associata:

$$S^I = \overline{S}I + S\overline{I}$$

Facciamo lo stesso per gli output:

Stato attuale (S)	Output(O)
0	1
1	0

$$O = \overline{S}$$

Se dovessimo realizzare un circuito che fa questo, ecco cosa dovremmo creare:

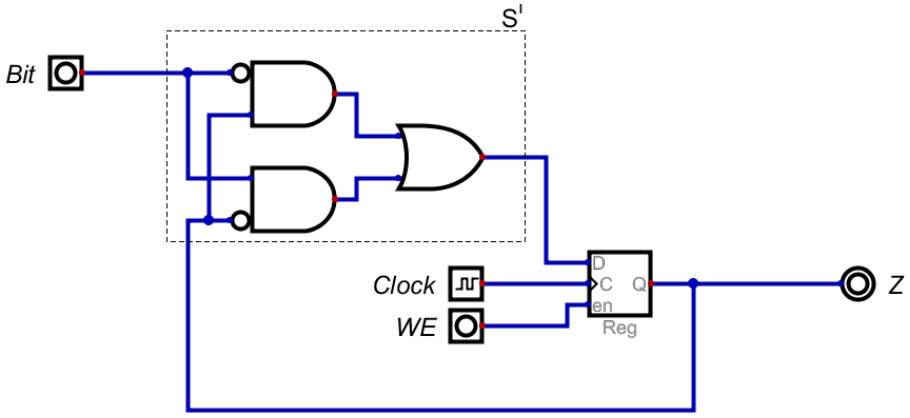


Figure 23: Circuito digitale dell'automa

I tempi legati al circuito sono:

- $S^I : 2\Delta t$
- $Z : 0$

Il tempo totale è quindi $2\Delta t$.
Riassumendo, per arrivare a capire come sviluppare una rete sequenziale tale, bisogna:

1. Capire il problema
2. Modellare un automa (Mealy o Moore)
3. Creare le tabelle di verità
4. Scrivere l'espressione logica annessa
5. Modellare il circuito digitale
6. Calcolare i tempi

8 Esercizi

8.1 Cosa calcola una rete

8.1.1 Descrizione

Dato una rete:

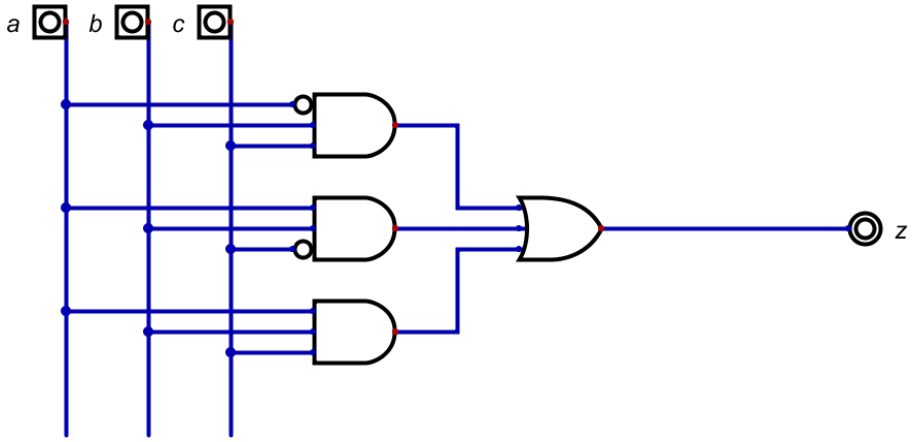


Figure 24: Rete sequenziale — Esercizio 1

Trovare:

1. Cosa calcola la rete.
2. Se è possibile un modo più efficiente di calcolarla.

8.1.2 Soluzione

Per trovare cosa calcola la rete, bisogna trovare l'**espressione** logica:

$$z = \overline{a}bc + ab\overline{c} + abc$$

Per capire invece se si può semplificare, bisogna provare in diversi modi:

Algebra booleana : si procede con l'effettuare raccoglimenti e semplificazioni.
In questo caso, si può aggiungere un quarto termine "abc" perche fare l'OR tra due valori uguali non cambia il risultato.

$$z = \overline{a}bc + ab\overline{c} + abc + abc$$

Adesso possiamo raccogliere:

$$z = (\overline{a} + a)bc + (\overline{c} + c)ab$$

$$z = bc + ab$$

$$(a + c)b$$

Come si può vedere si è semplificato molto.

Mappe di Kernaugh : si crea la tabella associata:

$a\backslash bc$	00	01	11	10
0	0	0	1	0
1	0	0	1	1

Si calcola l'espressione tramite i gruppi presi:

$$z = bc + ab = (a + c)b$$

8.2 Circuito di una sveglia

8.2.1 Descrizione

Un orologio digitale rappresenta le cifre necessarie per segnare il tempo con un totale di 7 segmenti che possono essere accesi o spenti a seconda della cifra rappresentata, di solito si nominano in questo ordine:

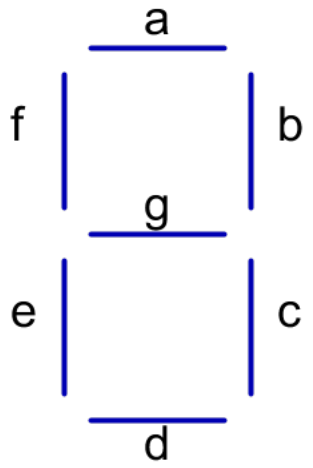


Figure 25: Segmenti orologio

Trovare l'espressione logica associata ai segmenti dell'orologio.

8.2.2 Soluzione

Iniziamo col dire che le cifre vanno da 0 a 9 e per rappresentare questi numeri in binario ci servono 4 bit.
Poi diciamo che ogni segmento (lettera):

$$segmento = \begin{cases} 1, & \text{segmento utilizzato} \\ 0, & \text{altrimenti} \end{cases}$$

Adesso si svolge la tabella di verità, la quale è molto grande:

Cifra	x_3	x_2	x_1	x_0	a	b	c	d	e	f	g
0	0	0	0	0	1	1	1	1	1	1	0
1	0	0	0	1	0	1	1	0	0	0	0
2	0	0	1	0	1	1	0	1	1	0	1
3	0	0	1	1	1	1	1	1	0	0	1
4	0	1	0	0	0	1	1	0	0	1	1
5	0	1	0	1	1	0	1	1	0	1	1
6	0	1	1	0	1	0	1	1	1	1	1
7	0	1	1	1	1	1	1	0	0	0	0
8	1	0	0	0	1	1	1	1	1	1	1
9	1	0	0	1	1	1	1	1	0	1	1
10	1	0	1	0	X	X	X	X	X	X	X
11	1	0	1	1	X	X	X	X	X	X	X
12	1	1	0	0	X	X	X	X	X	X	X
13	1	1	0	1	X	X	X	X	X	X	X
14	1	1	1	0	X	X	X	X	X	X	X
15	1	1	1	1	X	X	X	X	X	X	X

Come si può vedere molti casi sono nulli, infatti andrebbe fatto in modo che il circuito non accetti input con risultati nulli.

Per calcolare l'espressione (di ogni output(segmento)), si fa come al solito:

$$
\begin{aligned}
a = {} & \overline{x_3}\,\overline{x_2}\,\overline{x_1}\,\overline{x_0} + \overline{x_3}\,\overline{x_2}\,x_1\,\overline{x_0} \\
& + \overline{x_3}\,\overline{x_2}\,x_1\,x_0 + \overline{x_3}\,x_2\,\overline{x_1}\,x_0 \\
& + \overline{x_3}\,x_2\,x_1\,\overline{x_0} + \overline{x_3}\,x_2\,x_1\,x_0 \\
& + x_3\,\overline{x_2}\,\overline{x_1}\,\overline{x_0} + x_3\,\overline{x_2}\,\overline{x_1}\,x_0 \\
b = {} & \overline{x_3}\,\overline{x_2}\,\overline{x_1}\,\overline{x_0} + \overline{x_3}\,\overline{x_2}\,\overline{x_1}\,x_0 \\
& + \overline{x_3}\,\overline{x_2}\,x_1\,\overline{x_0} + \overline{x_3}\,\overline{x_2}\,x_1\,x_0 \\
& + \overline{x_3}\,x_2\,\overline{x_1}\,\overline{x_0} + \overline{x_3}\,x_2\,x_1\,x_0 \\
& + x_3\,\overline{x_2}\,\overline{x_1}\,\overline{x_0} + x_3\,\overline{x_2}\,\overline{x_1}\,x_0 \\
c = {} & \overline{x_3}\,\overline{x_2}\,\overline{x_1}\,\overline{x_0} + \overline{x_3}\,\overline{x_2}\,\overline{x_1}\,x_0 \\
& + \overline{x_3}\,\overline{x_2}\,x_1\,x_0 + \overline{x_3}\,x_2\,\overline{x_1}\,\overline{x_0} \\
& + \overline{x_3}\,x_2\,\overline{x_1}\,x_0 + \overline{x_3}\,x_2\,x_1\,\overline{x_0} \\
& + \overline{x_3}\,x_2\,x_1\,x_0 + x_3\,\overline{x_2}\,\overline{x_1}\,\overline{x_0} \\
& + x_3\,\overline{x_2}\,\overline{x_1}\,x_0 \\
d = {} & \overline{x_3}\,\overline{x_2}\,\overline{x_1}\,\overline{x_0} + \overline{x_3}\,\overline{x_2}\,x_1\,\overline{x_0} \\
& + \overline{x_3}\,\overline{x_2}\,x_1\,x_0 + \overline{x_3}\,x_2\,x_1\,x_0 \\
& + \overline{x_3}\,x_2\,\overline{x_1}\,\overline{x_0} + \overline{x_3}\,x_2\,x_1\,\overline{x_0} \\
& + x_3\,\overline{x_2}\,\overline{x_1}\,\overline{x_0} + x_3\,\overline{x_2}\,\overline{x_1}\,x_0 \\
e = {} & \overline{x_3}\,\overline{x_2}\,\overline{x_1}\,\overline{x_0} + \overline{x_3}\,\overline{x_2}\,x_1\,\overline{x_0} \\
& + \overline{x_3}\,x_2\,x_1\,\overline{x_0} + x_3\,\overline{x_2}\,\overline{x_1}\,\overline{x_0} \\
f = {} & \overline{x_3}\,\overline{x_2}\,\overline{x_1}\,\overline{x_0} + \overline{x_3}\,x_2\,\overline{x_1}\,\overline{x_0} \\
& + \overline{x_3}\,x_2\,x_1\,\overline{x_0} + x_3\,\overline{x_2}\,\overline{x_1}\,\overline{x_0} \\
& + x_3\,\overline{x_2}\,\overline{x_1}\,x_0 \\
g = {} & \overline{x_3}\,\overline{x_2}\,x_1\,\overline{x_0} + \overline{x_3}\,\overline{x_2}\,x_1\,x_0 \\
& + \overline{x_3}\,x_2\,\overline{x_1}\,\overline{x_0} + \overline{x_3}\,x_2\,\overline{x_1}\,x_0 \\
& + \overline{x_3}\,x_2\,x_1\,\overline{x_0} + x_3\,\overline{x_2}\,\overline{x_1}\,\overline{x_0} \\
& + x_3\,\overline{x_2}\,\overline{x_1}\,x_0
\end{aligned}
$$

L'esercizio è simile al precedente, ovviamente più ampio e complesso.